

Adaptive HNSW Search via Entry-Point Topology and Aggregate Recall Curves

Algorithm & Logic Specification

1 Introduction

In Hierarchical Navigable Small World (HNSW) graphs, the **ef** (exploration factor) parameter strictly controls the trade-off between search speed and recall. A constant **ef** across all queries is computationally inefficient due to the vast variance in query difficulty. The **2Metric Adaptive EF** architecture solves this by extracting structural signals from an initial low-cost graph probe. These signals are mapped to a calibrated *Aggregate Recall Curve* table, assigning the optimal **ef** dynamically on a per-query basis.

2 The Dual-Metric Estimator

The estimator isolates the two primary causes of search failure in HNSW: global routing errors (upper layers) and local hub entrapment (base layer).

2.1 Drop-off Entry Point Distance (d_{ep}) – Global Routing Signal

The d_{ep} metric measures the absolute distance from the query to the final node of the upper-layer greedy descent (the exact point where the search drops into Layer 0).

$$d_{ep} = \text{dist}(q, \text{Layer}_0 \text{ Entry Node}) \quad (1)$$

- **High d_{ep} :** The upper layers were sparse and routed poorly, dropping the search far away from the true nearest neighbors. Layer 0 will require a massive exhaustive crawl to recover. Hard query.
- **Low d_{ep} :** The upper layers successfully dropped the search precisely in the target neighborhood. Easy query.

2.2 Topological Rank Entrapment (RV) – Local Topological Signal

RV measures the degree to which the Layer-0 search wastes iterations revisiting identical hub nodes, weighted heavily toward early encounters via exponential decay (γ).

$$RV = \frac{\sum_{i=1}^N \mathbf{I}(e_i \text{ is revisit}) \cdot e^{-\gamma \cdot (i/N)}}{\sum_{i=1}^N e^{-\gamma \cdot (i/N)}} \quad (2)$$

- **High RV :** The local neighborhood is entangled with massive hubs. Hard query.
- **Low RV :** The graph structure leads linearly to the target. Easy query.

3 Algorithms and Pseudocode

3.1 Phase 1: The Probe Phase

Algorithm 1: HNSW Dual-Metric Probe Extraction

Input: HNSW Graph G , Query q , Probe Cap C , Decay γ
Output: Estimated Signals (d_{ep}, RV)

```

1  $ep \leftarrow \text{GreedyDescent}(G_{upper}, q)$  // Drop to Layer 0
2  $d_{ep} \leftarrow \text{dist}(q, ep)$ 
3  $visited \leftarrow \{ep\}$ ,  $candidates \leftarrow \{ep\}$ ,  $edges \leftarrow []$ 
4 while  $candidates \neq \emptyset$  and  $|evaluated| \leq C$  do
5    $curr \leftarrow candidates.pop\_closest()$ 
6   for  $nbr \in neighbors(curr)$  do
7      $d \leftarrow \text{dist}(q, nbr)$ 
8      $is\_revisit \leftarrow nbr \in visited$ 
9      $edges.append((d, is\_revisit))$ 
10    if not  $is\_revisit$  then
11       $visited.insert(nbr)$ ;  $candidates.push(nbr)$ 
12    end
13  end
14 end
15 Sort  $edges$  by  $d$  ascending
16  $W_{tot} \leftarrow 0$ ,  $W_{vis} \leftarrow 0$ ,  $N \leftarrow length(edges)$ 
17 for  $i \leftarrow 1$  to  $N$  do
18    $w \leftarrow \exp(-\gamma \cdot (i/N))$ 
19    $W_{tot} \leftarrow W_{tot} + w$ 
20   if  $edges[i].is\_revisit$  then
21      $W_{vis} \leftarrow W_{vis} + w$ 
22   end
23 end
24  $RV \leftarrow W_{vis} / \max(W_{tot}, \epsilon)$ 
25 return  $(d_{ep}, RV)$ 

```

3.2 Phase 2: Table Calibration via Aggregate Recall Curves

Rather than recording individual required EFs and suffering from high within-bin variance, the table generator groups training queries into a 32×32 grid of (d_{ep}, RV) bins. It steps through ef values using an **Inverse-Slope Dynamic Increment** until the *aggregate average recall* of the entire bin satisfies the target recall constraint.

To save computational overhead, an early-stopping ϵ -threshold aborts evaluations for any query that flatlines mathematically.

Algorithm 2: Dynamic Aggregate Curve Calibration

Input: Query Stream Q , Graph G , Target Recall T_R , Min Step k_{half} **Output:** 2D Grid Mapping $(d_{ep}, RV) \rightarrow \text{Curve}(ef, avg_recall)$

```
1 for each bin  $B \in \text{Grid}(d_{ep}, RV)$  do
2    $ef_1 \leftarrow k, ef_2 \leftarrow k \times 1.5$ 
3    $rec_1 \leftarrow \text{AvgRecall}(B, ef_1)$ 
4    $rec_2 \leftarrow \text{AvgRecall}(B, ef_2)$ 
5    $B.curve \leftarrow [(ef_1, rec_1), (ef_2, rec_2)]$ 
6    $latest\_ef \leftarrow ef_2, prev\_ef \leftarrow ef_1$ 
7    $latest\_rec \leftarrow rec_2, prev\_rec \leftarrow rec_1$ 
8    $ef\_diff \leftarrow latest\_ef - prev\_ef$ 
9   while  $T_R - latest\_rec > 1e^{-4}$  and  $latest\_ef < max\_ef$  do
10     $recall\_diff \leftarrow latest\_rec - prev\_rec$ 
11    if  $recall\_diff < 1e^{-6}$  then
12      break
13    end
14    // Proportional Guess using Inverse-Slope
15     $step \leftarrow \text{int} \left( ef\_diff \cdot \frac{T_R - latest\_rec}{recall\_diff} \right)$ 
16     $ef\_diff \leftarrow \max(step, k_{half})$  // Strict floor guard
17     $next\_ef \leftarrow \min(latest\_ef + ef\_diff, max\_ef)$ 
18     $next\_rec \leftarrow \text{AvgRecall}(B, next\_ef)$ 
19     $B.curve.push(next\_ef, next\_rec)$ 
20     $prev\_ef \leftarrow latest\_ef, prev\_rec \leftarrow latest\_rec$ 
21     $latest\_ef \leftarrow next\_ef, latest\_rec \leftarrow next\_rec$ 
22 end
```

3.3 Phase 3: The Adaptive Search Loop with Query-Weighted Floor

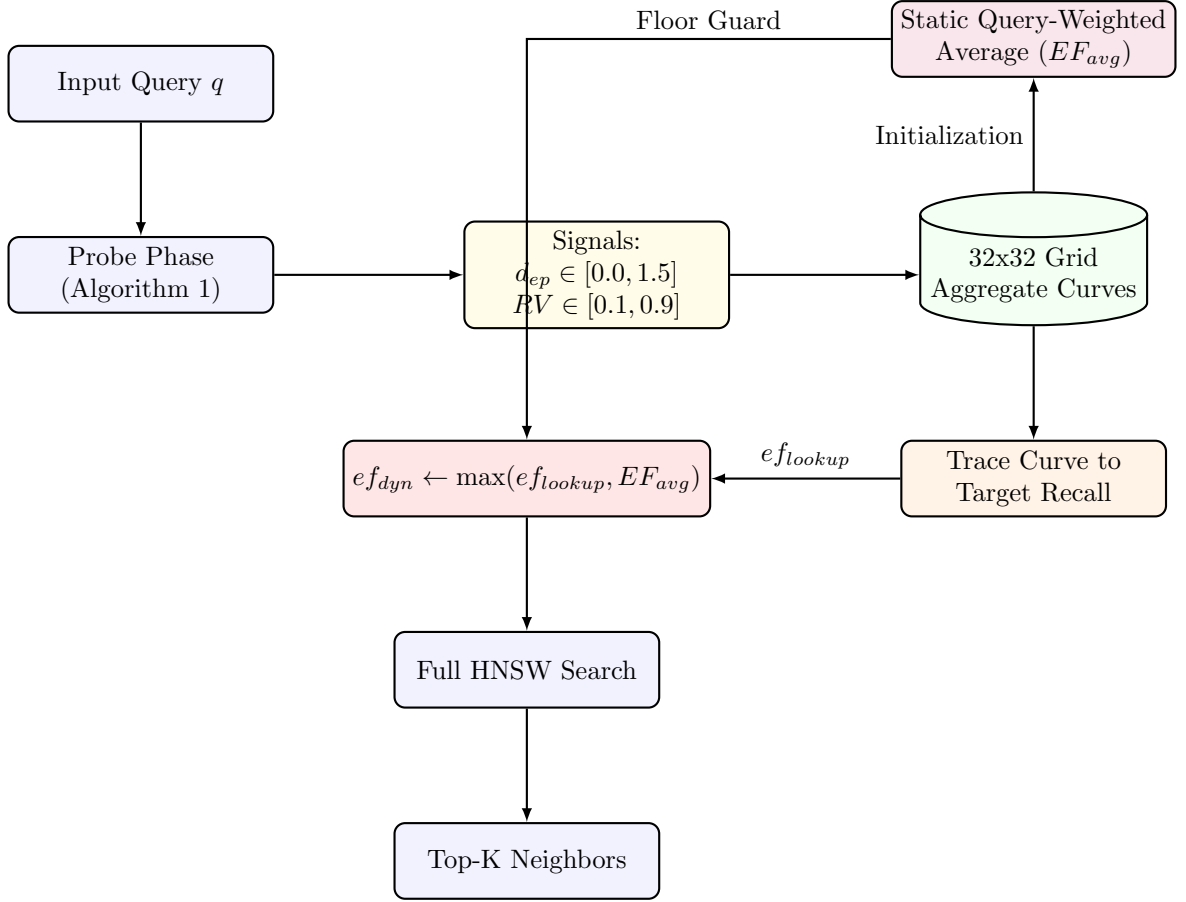
To prevent artificial inflation via cumulative feedback loops, the online search utilizes a **Static Query-Weighted Table Average** as a safe floor mechanism. The overall required ef across all valid curves in the lookup table is averaged, ensuring easy queries never drop beneath the dataset’s topological baseline.

Algorithm 3: Online Adaptive EF Search Loop

Input: Query q , Graph G , Lookup Table T , Table Average Floor EF_{avg} **Output:** Nearest Neighbors for q

```
1  $(d_{ep}, RV) \leftarrow \text{ProbeExtraction}(G, q)$ 
2  $ef_{lookup} \leftarrow T.\text{find\_minimum\_ef\_for\_target\_recall}(d_{ep}, RV)$ 
3 // Floor isolated strictly to static map baseline
4  $ef_{dyn} \leftarrow \max(ef_{lookup}, EF_{avg})$ 
5  $ef_{dyn} \leftarrow \max(ef_{dyn}, k)$ 
6  $G.setEf(ef_{dyn})$ 
7  $results \leftarrow G.searchKnn(q)$ 
8 return results
```

4 System Architecture Visualization



5 Conclusion

By utilizing d_{ep} (measuring structural routing failures) orthogonally against RV (measuring local network entanglement), the 2Metric system successfully splits topological difficulty into its two physical causes. The resulting Aggregate Recall Curves smoothly interpolate required limits, while the Query-Weighted Floor safely halts runaway error spiraling, enabling near-perfect target recall achievement with significantly reduced average computation.